

PROYECTO PEDAGÓGICO INTEGRADOR
PROGRAMA INGENIERÍA EN SOFTWARE
SEGUNDO SEMESTRE

DIAGNOSTICO ORGANIZACIONAL: GLOBANT COLOMBIA

ELABORADO POR

Samuel David Colmenárez Quero

Jairo Yeison Sanchez Lopez

Ashley Yulieth Mendez Romero

TECNOLÓGICO DE ANTIOQUIA INSTITUCIÓN UNIVERSITARIA

FACULTAD DE INGENIERÍA

ÁREA INFORMÁTICA

MEDELLÍN

2025

REGISTRO DE ACTUALIZACIONES

El registro de actualizaciones permite el control de las versiones del informe del proyecto. Cada vez que se realiza una actualización se debe registrar: la fecha, la versión que está modificando del informe, la descripción de la actualización y el integrante del equipo de trabajo que la realizó.

Fecha	Versión	Descripción de la actualización	Autor
01/05/2025	1.0	Versión inicial del informe	Jairo, Ashley, Samuel
08/05/2025	1.1	Versión inicial punto uno del informe	Jairo, Ashley, Samuel
11/05/2025	1.2	Versión inicial punto dos del informe (Correcciones)	Jairo, Ashley, Samuel
08/06/2025	2.0	Versión segunda del informe	Jairo, Ashley
23/11/2025	3.0	Versión tercera del informe	Jairo, Ashley, Samuel

INTRODUCCIÓN

En el entorno empresarial actual, donde el cambio es constante y la competencia cada vez más exigente, las organizaciones se ven en la necesidad de adaptarse rápidamente y adoptar metodologías que les permitan ser más ágiles y eficientes. En este contexto, Globant Colombia, reconocida por su enfoque innovador en el desarrollo de soluciones tecnológicas, ha incorporado la gestión ágil de proyectos como un pilar clave dentro de sus procesos internos.

Sin embargo, a pesar de contar con marcos ágiles bien definidos, se han identificado retos recurrentes en la planificación, ejecución y seguimiento de los proyectos. Estos desafíos han afectado la eficiencia operativa y, en consecuencia, la capacidad de entregar valor de manera oportuna a los clientes.

Este proyecto tiene como propósito principal realizar un diagnóstico organizacional enfocado específicamente en la gestión de proyectos ágiles en Globant Colombia. A través de un análisis detallado del proceso actual, se han detectado debilidades como la falta de trazabilidad, la escasa visibilidad del progreso de los proyectos y la dispersión de herramientas utilizadas. Estos factores están generando dificultades en la coordinación entre equipos y limitando la toma de decisiones estratégicas.

Para hacer frente a esta situación, se propone desarrollar una aplicación web personalizada que permita centralizar toda la información relacionada con los proyectos. Esta herramienta no solo facilitará la gestión de tareas y el seguimiento en tiempo real, sino que también promoverá una comunicación más fluida y efectiva entre los miembros del equipo.

La aplicación será diseñada bajo principios ágiles y adaptada a las necesidades específicas de la organización, con el objetivo de impulsar la productividad y asegurar que cada proyecto avance de manera eficiente hacia el cumplimiento de sus metas.

TABLA DE CONTENIDO

REGISTRO DE ACTUALIZACIONES	2
INTRODUCCIÓN.....	3
TABLA DE CONTENIDO.....	4
LISTA DE TABLA E ILUSTRACIONES	8
1. CONTEXTO DEL SISTEMA.	9
1.1. Nombre de la organización.	9
1.2. Caracterización de la organización.	9
1.3. Misión.	13
1.4. Visión.....	13
1.5. Organigrama.	15
2. CONTEXTO DEL SISTEMA ACTUAL.....	16
2.1. Nombre del proceso organizacional.....	16
2.2. Descripción del proceso organizacional.	16
2.3. Problemática del proceso.	19
3. PROPUESTA DE SOLUCIÓN.....	22
3.1. Descripción del sistema software.....	22
3.2. Funcionalidades propuestas.	24

3.3.	Desarrollo de la propuesta en Python	26
	DICCIONARIOS Y LISTAS	26
	FUNCIONALIDAD 1 LOGIN.....	26
	FUNCIONALIDAD 2 MENU DEL ADMINISTRADOR.....	27
	FUNCIONALIDAD 3 REGISTRO DE CLIENTES	28
	FUNCIONALIDAD 4 VER LOS CLIENTES.....	30
	FUNCIONALIDAD 5 REGISTRAR USUARIOS (SCRUM MASTER Y PROGRAMADORES).....	31
	FUNCIONALIDAD 6 MENU DEL CLIENTE	32
	FUNCIONALIDAD 7 MENU SCRUM MASTER.....	34
	FUNCIONALIDAD 8 MENU PROGRAMADOR	35
	SISTEMA PRINCIPAL Y CAMBIO DE USUARIO	37
3.4	Desarrollo de la propuesta en Java.....	38
	PATRON BUILDER	38
	Justificación.....	38
	PATRON FACTORY	39
	Explicación.....	39
	PATRON SIGLETON	39
	Explicación.....	40

CLASE PRINCIPAL	41
Explicación.....	41
HERENCIA, POLIMORFISMO Y ENCAPSULAMIENTO	41
Clase ClienteGlobant (Herencia y Polimorfismo)	41
Explicación.....	42
Clase LiderGlobant (Polimorfismo).....	42
Explicación.....	43
Clase MenuGlobant (Polimorfismo Dinámico)	43
Explicación.....	45
Clase PersonaGlobant (Herencia y Encapsulamiento).....	45
Explicación.....	46
Clase ProgramadorGlobant (Polimorfismo)	47
Explicación.....	47
Clase RolGloant (Enumeración)	47
Explicación.....	48
Clase TareaGlobant (Encapsulamiento).....	48
Explicación.....	49
DIAGRAMAS.....	50
.....	50
1.1. Conclusiones	55

2.	Referencias.....	56
----	------------------	----

LISTA DE TABLA E ILUSTRACIONES

Ilustración 1 Diagrama de Clases	50
Ilustración 2 Diagrama de Estados - TareaGlobant.....	51
Ilustración 3 Diagrama de Estados - ServicioAuthGlobant	52
Ilustración 4 Diagrama de Uso – Sistema de Globant.....	53
Ilustración 5 Diagrama de secuencia – Proceso imaginario de tareas.....	54

1. CONTEXTO DEL SISTEMA.

1.1. Nombre de la organización.

Organización Globant Colombia.

1.2. Caracterización de la organización.

Globant, una compañía nacida en Argentina en 2003, se ha convertido en uno de los grandes jugadores globales en el mundo de la tecnología y el desarrollo de software. Desde que llegó a Colombia en 2011, su presencia ha crecido de forma constante, posicionándose como una de las firmas más influyentes en el ecosistema digital del país. Con oficinas en Bogotá, Medellín y Cali, Globant Colombia ha sabido combinar talento local con innovación internacional para impulsar proyectos de gran escala, tanto a nivel nacional como para marcas globales.

Una de las principales razones detrás del éxito de Globant en Colombia es su capacidad para atraer y retener talento. La empresa cuenta hoy con miles de colaboradores, conocidos como “Globers”, que trabajan en áreas tan diversas como desarrollo de software, análisis de datos, inteligencia artificial, diseño de experiencias de usuario, ciberseguridad y soluciones en la nube. Lo interesante es que muchos de estos profesionales participan en proyectos para clientes internacionales sin necesidad de salir del país, lo que convierte a Globant en una opción muy atractiva para los jóvenes talentos colombianos que buscan desafíos globales.

La cultura organizacional es un sello distintivo de la empresa. Globant apuesta por una estructura horizontal, donde se fomenta la autonomía, la innovación y el trabajo en equipo. Los equipos se organizan en “Studios”, que son grupos especializados en distintas tecnologías

o industrias, permitiendo que cada proyecto cuente con expertos que realmente entienden las necesidades del cliente. Además, la empresa impulsa constantemente programas de formación interna, como Globant University, para asegurarse de que sus colaboradores estén siempre al día con las últimas tendencias tecnológicas.

No menos relevante es el esfuerzo que Globant hace por promover la diversidad y la inclusión. Iniciativas como “Women That Build” buscan abrir espacios para las mujeres en un sector que históricamente ha sido dominado por hombres, mostrando que la compañía no solo está interesada en crecer como negocio, sino también en generar un impacto social positivo.

Desde el punto de vista económico, Globant ha sido clave para dinamizar el sector tecnológico colombiano. Además de generar miles de empleos calificados, ha ayudado a posicionar a Colombia como un destino atractivo para la inversión extranjera, particularmente en el sector de servicios digitales. Esto se debe en parte a factores como la calidad del talento local, la cercanía cultural y geográfica con mercados como el estadounidense, y un ecosistema tecnológico que ha madurado notablemente en los últimos años.

Pero Globant no se detiene ahí. La empresa también apuesta por la innovación constante, invirtiendo en áreas emergentes como el blockchain, el metaverso, la inteligencia artificial y la sostenibilidad digital. Esto no solo beneficia a sus clientes, que reciben soluciones de última generación, sino que también permite a sus equipos crecer profesionalmente y participar en proyectos que tienen el potencial de transformar industrias enteras.

Finalmente, hay que destacar el papel que Globant ha asumido en el ecosistema local. A través de alianzas con universidades, gobiernos y startups, la compañía ha contribuido a fortalecer el tejido tecnológico del país, apoyando el crecimiento del emprendimiento digital y ayudando a consolidar a Colombia como uno de los líderes en innovación en América Latina.

En definitiva, Globant Colombia representa mucho más que una empresa tecnológica: es un actor central en la transformación digital del país, un puente entre el talento colombiano y los mercados globales, y un motor de innovación que sigue empujando los límites de lo posible. Su historia en Colombia es la historia de cómo una empresa puede crecer, innovar y generar impacto al mismo tiempo.

Globant, una compañía nacida en Argentina en 2003, se ha convertido en uno de los grandes jugadores globales en el mundo de la tecnología y el desarrollo de software. Desde que llegó a Colombia en 2011, su presencia ha crecido de forma constante, posicionándose como una de las firmas más influyentes en el ecosistema digital del país. Con oficinas en Bogotá, Medellín y Cali, Globant Colombia ha sabido combinar talento local con innovación internacional para impulsar proyectos de gran escala, tanto a nivel nacional como para marcas globales.

Una de las principales razones detrás del éxito de Globant en Colombia es su capacidad para atraer y retener talento. La empresa cuenta hoy con miles de colaboradores, conocidos como “Globers”, que trabajan en áreas tan diversas como desarrollo de software, análisis de datos, inteligencia artificial, diseño de experiencias de usuario, ciberseguridad y soluciones en la nube. Lo interesante es que muchos de estos profesionales participan en proyectos para clientes internacionales sin necesidad de salir del país, lo que convierte a Globant en una opción muy atractiva para los jóvenes talentos colombianos que buscan desafíos globales.

La cultura organizacional es un sello distintivo de la empresa. Globant apuesta por una estructura horizontal, donde se fomenta la autonomía, la innovación y el trabajo en equipo. Los equipos se organizan en “Studios”, que son grupos especializados en distintas tecnologías o industrias, permitiendo que cada proyecto cuente con expertos que realmente entienden las

necesidades del cliente. Además, la empresa impulsa constantemente programas de formación interna, como Globant University, para asegurarse de que sus colaboradores estén siempre al día con las últimas tendencias tecnológicas.

No menos relevante es el esfuerzo que Globant hace por promover la diversidad y la inclusión. Iniciativas como “Women That Build” buscan abrir espacios para las mujeres en un sector que históricamente ha sido dominado por hombres, mostrando que la compañía no solo está interesada en crecer como negocio, sino también en generar un impacto social positivo.

Desde el punto de vista económico, Globant ha sido clave para dinamizar el sector tecnológico colombiano. Además de generar miles de empleos calificados, ha ayudado a posicionar a Colombia como un destino atractivo para la inversión extranjera, particularmente en el sector de servicios digitales. Esto se debe en parte a factores como la calidad del talento local, la cercanía cultural y geográfica con mercados como el estadounidense, y un ecosistema tecnológico que ha madurado notablemente en los últimos años.

Pero Globant no se detiene ahí. La empresa también apuesta por la innovación constante, invirtiendo en áreas emergentes como el blockchain, el metaverso, la inteligencia artificial y la sostenibilidad digital. Esto no solo beneficia a sus clientes, que reciben soluciones de última generación, sino que también permite a sus equipos crecer profesionalmente y participar en proyectos que tienen el potencial de transformar industrias enteras.

Finalmente, hay que destacar el papel que Globant ha asumido en el ecosistema local. A través de alianzas con universidades, gobiernos y startups, la compañía ha contribuido a fortalecer el tejido tecnológico del país, apoyando el crecimiento del emprendimiento digital y ayudando a consolidar a Colombia como uno de los líderes en innovación en América Latina.

En definitiva, Globant Colombia representa mucho más que una empresa tecnológica: es un actor central en la transformación digital del país, un puente entre el talento colombiano y los mercados globales, y un motor de innovación que sigue empujando los límites de lo posible. Su historia en Colombia es la historia de cómo una empresa puede crecer, innovar y generar impacto al mismo tiempo.

1.3. Misión.

Nuestra misión en Globant Colombia es impulsar la transformación digital de organizaciones en todo el mundo, combinando talento, creatividad y tecnología de punta. Queremos acompañar a nuestros clientes en sus desafíos más ambiciosos, ayudándolos a reinventar sus negocios, optimizar procesos y crear experiencias únicas para sus usuarios. Lo hacemos apoyándonos en equipos multidisciplinarios, con un profundo conocimiento tecnológico y un compromiso genuino por entregar soluciones de alto impacto.

En Colombia, nuestra misión también es formar, inspirar y potenciar el talento local. Sabemos que nuestro mayor activo son las personas, por eso nos dedicamos a construir un entorno donde cada Globant pueda crecer profesional y personalmente, aprendiendo constantemente y participando en proyectos globales que dejan huella. Nos esforzamos por ser una empresa inclusiva, abierta a nuevas ideas, donde la diversidad es vista como una fuente de fortaleza y no de diferencia.

1.4. Visión.

En Globant Colombia, imaginamos un futuro en el que la tecnología no sea solo una herramienta, sino un verdadero puente para conectar ideas, personas y oportunidades. Queremos ser reconocidos como líderes en la creación de soluciones digitales innovadoras que impacten positivamente a empresas y comunidades, no solo en Colombia, sino en todo el mundo. Nuestra visión es construir un espacio donde el talento colombiano pueda crecer,

aprender y desarrollar su máximo potencial, trabajando en proyectos que rompen barreras y transforman industrias.

Vemos a Globant como un agente de cambio que no se conforma con seguir tendencias, sino que busca crearlas. Queremos impulsar un ecosistema donde la diversidad, la inclusión y la sostenibilidad sean pilares fundamentales, demostrando que es posible crecer como empresa mientras se genera un impacto social y ambiental positivo. Aspiramos a ser el referente en transformación digital en América Latina, llevando el sello colombiano a cada rincón donde tengamos presencia. Para nosotros, la innovación no es un fin, sino un camino constante, lleno de desafíos, aprendizajes y logros compartidos. Así, cada día trabajamos para construir el futuro que soñamos, un futuro digital, humano e inclusivo.

1.5. Organigrama.

Ilustración 1. Organigrama de la organización: Globant.



Nota: Organigrama jerárquico de la empresa Globant, tomado de una presentación de la empresa realizada en prezi.

<https://prezi.com/p/f3ov3tf1qwfr/globant/>

2. CONTEXTO DEL SISTEMA ACTUAL.

2.1. Nombre del proceso organizacional.

Gestión de Proyectos Ágiles

(utilizado para planificar, ejecutar y controlar proyectos TI)

2.2. Descripción del proceso organizacional.

Este proceso describe cómo se gestionan los proyectos de desarrollo de software en Globant Colombia utilizando metodologías ágiles (principalmente Scrum). Abarca desde la planificación inicial hasta la entrega continua de valor al cliente.

Tabla 1. Proceso Organizacional Actual.

Etapa	Actividad	Entradas	Salidas	Responsables
Inicio del proyecto	Reunión de Kick-Off con el cliente y equipo	Contrato, requerimientos iniciales, equipo asignado	Product Backlog inicial, acuerdos de trabajo	Scrum Master, Product Owner
Planificación del Sprint	Planificación del trabajo a realizar durante el sprint	Product Backlog priorizado	Sprint Backlog, objetivos del sprint	Product Owner, Equipo de desarrollo
Ejecución del Sprint	Desarrollo de funcionalidades según el Sprint Backlog	Tareas definidas, criterios de aceptación	Funcionalidades desarrolladas y testeadas	Equipo de desarrollo, QA
Daily Scrum	Reunión diaria de sincronización del equipo	Avances, impedimentos	Seguimiento del progreso, ajustes en la planificación	Scrum Master, Equipo de desarrollo

Revisión del Sprint	Demostración del trabajo completado al cliente	Incremento del producto, criterios de aceptación cumplidos	Retroalimentación del cliente, funcionalidades aprobadas o rechazadas	Product Owner, Cliente, Dev Team
Retrospectiva del Sprint	Evaluación del proceso para detectar mejoras	Experiencias del sprint	Lista de mejoras a aplicar en el próximo sprint	Scrum Master, Todo el equipo
Liberación / Entrega continua	Entrega del producto o actualización en entorno productivo	Funcionalidades terminadas, pruebas exitosas	Versión liberada del producto	DevOps, QA, Product Owner
Actualización del Backlog	Refinamiento del Product Backlog para próximos sprints	Retroalimentación del cliente, nuevas prioridades	Backlog actualizado	Product Owner

Nota: Tabla creada con datos de Blog Globant, sobre las Metodologías ágiles de desarrollo de software.

Estructura lógica.

1. Entradas generales del proceso:

- Requisitos del cliente
- Contrato o acuerdo de trabajo
- Backlog de producto

- Recursos humanos y técnicos

2. Salidas generales del proceso:

- Producto funcional incremental
- Reportes de avance
- Retroalimentación documentada
- Mejoras continuas implementada

2.3. Problemática del proceso.

Actualmente, el proceso de Gestión de Proyectos Ágiles en Globant Colombia enfrenta algunos desafíos que están impactando la eficiencia y la entrega oportuna de los productos a los clientes. Uno de los problemas más relevantes es la ausencia de una herramienta centralizada y automatizada que permita hacer un seguimiento efectivo de las tareas, los indicadores de desempeño y la comunicación entre los miembros del equipo.

Causas principales:

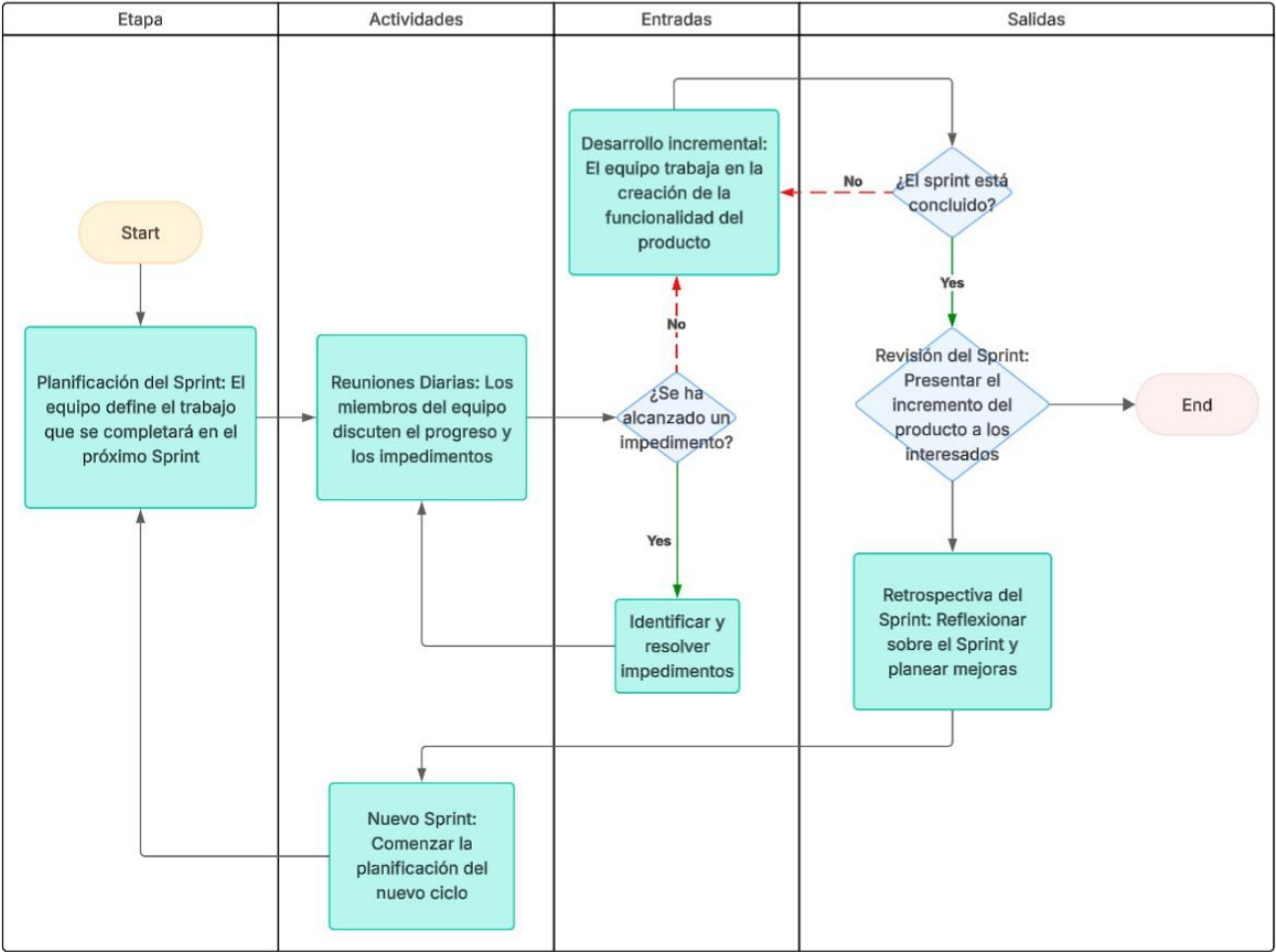
- **Uso disperso de múltiples plataformas** (como Jira, Trello, correos, hojas de cálculo), lo cual genera duplicidad de información y pérdida de trazabilidad.
- **Falta de integración entre herramientas de desarrollo y gestión**, dificultando el control en tiempo real del avance.
- **Dependencia excesiva de reuniones para obtener actualizaciones**, lo que reduce el tiempo efectivo de desarrollo.
- **Documentación del backlog y resultados del sprint poco estandarizada**, lo que complica la evaluación de métricas.

Consecuencias:

- **Retrasos en las entregas de sprint** por descoordinación o mala planificación.
- **Desmotivación del equipo** al no contar con visibilidad clara de sus tareas y objetivos.
- **Dificultad para tomar decisiones ágiles**, por falta de información consolidada y actualizada.
- **Menor satisfacción del cliente** debido a entregas incompletas o fuera del alcance esperado.

Este diagnóstico evidencia la necesidad de implementar un sistema de información centralizado que automatice, estandarice y facilite la gestión ágil de proyectos dentro de la organización.

Ilustración 2. Diagrama de flujo proceso actual.



Nota: Diagrama de flujo del proceso actual de gestión de desarrollo de proyectos creado en https://lucid.app/lucidchart/a4a956e2-0588-4ae2-9a63-13fa6ff0e92d/edit?invitationId=inv_43ab9f66-8697-4adb-8190-4fec1d7b0467&page=0_0#

3. PROPUESTA DE SOLUCIÓN.

3.1. Descripción del sistema software.

El sistema propuesto es una aplicación web diseñada para optimizar el proceso de Gestión de Proyectos Ágiles, permitiendo a los equipos de desarrollo planificar, ejecutar y hacer seguimiento a proyectos en tiempo real desde cualquier ubicación y dispositivo con conexión a Internet.

Este sistema está basado en una arquitectura cliente-servidor con diseño responsivo, garantizando su correcto funcionamiento en navegadores modernos (Chrome, Firefox, Edge) y compatibilidad con sistemas operativos comunes (Windows, macOS, Linux, Android e iOS).

Características técnicas del sistema:

Accesibilidad

- Cumplimiento con estándares WCAG 2.1 para garantizar acceso a usuarios con discapacidades.
- Navegación mediante teclado y soporte para lectores de pantalla.
- Diseño responsive para adaptarse a distintos tamaños de pantalla (PC, tabletas, móviles).

Compatibilidad

- Compatible con los navegadores más utilizados del mercado.
- Integración con herramientas externas como GitHub, GitLab, Slack, y APIs R
- Soporte para múltiples idiomas mediante internacionalización (i18n).

Seguridad

- Autenticación y autorización mediante JWT y OAuth 2.0.
- Cifrado de datos sensibles usando HTTPS y TLS.
- Protección contra ataques comunes como XSS, CSRF, y SQL Injection.
- Gestión de roles y permisos para acceso segmentado.

Rendimiento

- Carga eficiente de datos usando peticiones asincrónicas (AJAX o fetch).
- Almacenamiento en caché para mejorar la velocidad de respuesta.
- Escalabilidad horizontal mediante arquitectura basada en microservicios o contenedores (Docker/Kubernetes).
- Uso de bases de datos relacionales (PostgreSQL/MySQL) o NoSQL (MongoDB), según necesidad.

Usabilidad (UI/UX)

- Interfaz intuitiva, amigable y minimalista, enfocada en la productividad.
- Experiencia de usuario optimizada para facilitar la colaboración en tiempo real.
- Panel de control con estadísticas, burndown charts, backlog dinámico y tableros Kanban.
- Feedback visual inmediato ante acciones del usuario (por ejemplo, animaciones, colores de estado, validaciones en tiempo real).

Mantenimiento y escalabilidad

- Código modular y reutilizable, siguiendo principios SOLID.
- Documentación técnica y de usuario.

- Arquitectura preparada para nuevas funcionalidades y escalamiento progresivo.

Este sistema proporcionará una solución integral que responde a las necesidades reales de los equipos de desarrollo, mejorando el control, la comunicación y la entrega continua de valor en cada sprint.

3.2. Funcionalidades propuestas.

En Antecedentes de aplicaciones web similares

1. Jira Software

- Fecha: 2002
- Autores: Atlassian
- Descripción: Plataforma ampliamente utilizada para la planificación, seguimiento y gestión de proyectos ágiles (Scrum, Kanban).

Funcionalidades principales:

- Creación y gestión de issues (tareas, errores, historias).
- Tableros Kanban y Scrum.
- Seguimiento de sprints y reportes (burndown, velocity).
- Integración con herramientas como Bitbucket, Confluence, GitHub.
- Permisos por rol y notificaciones automatizadas.

2. Trello

- Fecha: 2011
- Autores: Fog Creek Software (adquirida por Atlassian)
- Descripción: Aplicación web basada en tableros visuales tipo Kanban, orientada a la gestión colaborativa de tareas.

Funcionalidades principales:

- Tarjetas personalizables con checklists, etiquetas y fechas.
- Arrastrar y soltar tareas en columnas.
- Integración con calendarios y Slack.
- Automatización con reglas y botones (Butler).

3. ClickUp

- Fecha: 2017
- Autores: Zeb Evans y Alex Yurkowski
- Descripción: Plataforma de productividad todo-en-uno que combina gestión de tareas, documentación, seguimiento de tiempo y colaboración.

Funcionalidades principales:

- Múltiples vistas de proyecto (lista, Gantt, tablero).
- Documentación y wikis internas.
- Seguimiento de tiempo y objetivos.
- Integraciones con herramientas externas (Zapier, GitHub, Google Workspace).

Funcionalidades propuestas de la aplicación web

Con base en el análisis de los antecedentes y adaptado a las necesidades específicas del proceso organizacional de Globant, se proponen las siguientes funcionalidades:

Funcionalidad propuesta

1. Registro y autenticación de usuarios con control de roles (Scrum Master, Dev, PO, Cliente).
2. Creación y gestión de proyectos con definición de backlog inicial.
3. Planificación de sprints con asignación de tareas, prioridad, fechas y estimaciones.
4. Tablero visual tipo Kanban para seguimiento del estado de las tareas (To Do, Doing, Done).
5. Registro automático de tiempo y avance por tarea.
6. Notificaciones inteligentes sobre tareas asignadas, plazos y cambios.
7. Reportes ágiles como burndown chart, velocidad del equipo y cumplimiento de objetivos.
8. Módulo de retrospectiva con entradas colaborativas anónimas o identificadas.
9. Chat o sección de comentarios por tarea para comunicación interna del equipo.
10. Integración con repositorios de código (GitHub/GitLab) para vincular commits con tareas.

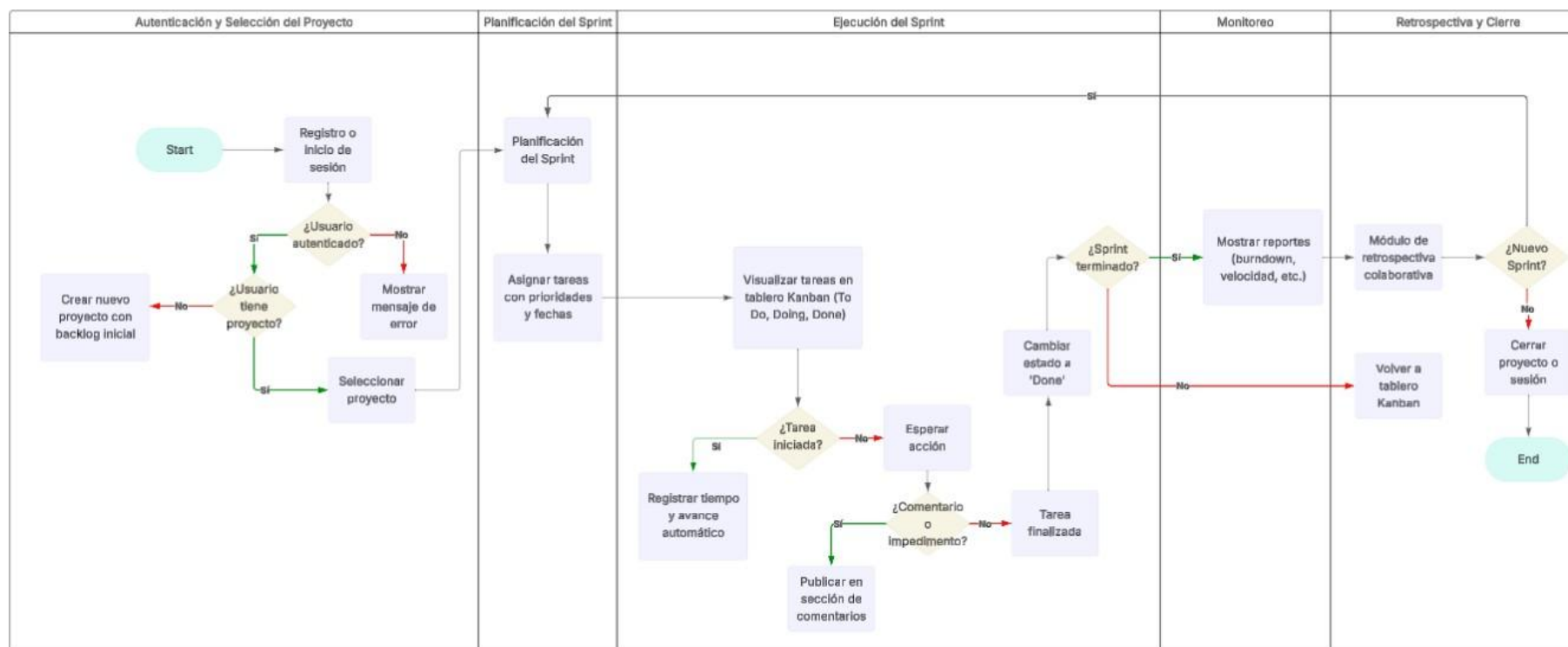
Aspectos diferenciadores del sistema propuesto

1. Interfaz personalizable y minimalista, adaptada a las necesidades reales del equipo (sin sobrecarga visual).
2. Asistente inteligente de planificación, que sugiere tareas según velocidad

histórica del equipo.

3. Historial completo de cada tarea, con trazabilidad de cambios, responsables y comentarios.
4. Modo cliente, que permite a los stakeholders externos ver avances sin modificar tareas.
5. Funcionalidad offline limitada, para registrar avances incluso sin conexión (sincronización posterior).

Ilustración 3. Diagrama de flujo proceso propuesto.



Nota: Diagrama de flujo creado en https://lucid.app/lucidchart/a4a956e2-0588-4ae2-9a63-13fa6ff0e92d/edit?invitationId=inv_43ab9f66-8697-4adb-8190-4fec1d7b0467&page=0_0#

Hipervínculo: <https://i.imgur.com/OCaeH6g.png>

3.3. Desarrollo de la propuesta en Python

DICCIONARIOS Y LISTAS

```
C: > Users > usuario > Music > globant.py > login.py > ...
1  #diccionario usuarios
2  usuarios = {
3      "admin_globant": {"contraseña": "admin", "rol": "admin"},
4      "scrum_master": {"contraseña": "scrum", "rol": "scrum_master"},
5      "desarrollador": {"contraseña": "desarrollo", "rol": "desarrollador"}
6  }
7
8  clientes = []
9  comentarios = {}
10 proyectos = {}
11 novedades = {}
12
```

Descripción: Se muestra el código fuente de las funciones clave que gestionan la autenticación de usuario.

FUNCIONALIDAD 1 LOGIN

```
13 #login
14 def login():
15     usuario = input("Ingresa el usuario: ")
16     contraseña = input("Ingresa tu contraseña: ")
17
18     if usuario in usuarios:
19         if contraseña == usuarios[usuario]["contraseña"]:
20             print("Bienvenido")
21             return usuarios[usuario]["rol"], usuario
22         else:
23             print("Contraseña incorrecta")
24     else:
25         print("Usuario no existe")
26     return None, None
27
```

Descripción: Se muestra la función login para agregar las credenciales al usuario y validar su acceso. Si las credenciales son correctas, retorna el rol y el nombre de usuario, de lo contrario informa los errores de usuario o contraseña.

```
PRO | Open folder in new window (ctrl + click) | TERMINAL | PORTS
PS C:\Users\usuario> & c:/Users/usuario/AppData/Local/Programs/Python/Python313/python.exe c:/Users/usuario/Music/globant.py/login.py
SISTEMA DE LOGIN
Ingresa el usuario: admin_globant
Ingresa tu contraseña: admin
Bienvenido
Bienvenido administrador Globant
```

Terminal del sistema del login

FUNCIONALIDAD 2 MENU DEL ADMINISTRADOR

```
#Menú Administrador
def menu_administrador():
    while True:
        print("Bienvenido administrador Globant")
        print("1. Registrar Cliente")
        print("2. Ver Clientes")
        print("3. Registrar Scrum Master o Programador")
        print("4. Cerrar sesión")

        opcion = input("Seleccione una opción: ")

        if opcion == "1":
            registrar_cliente()
        elif opcion == "2":
            ver_clientes()
        elif opcion == "3":
            registrar_usuario()
        elif opcion == "4":
            break
        else:
            print("Opción no válida")
```

Descripción: Se muestra el menú interactivo al usuario administrador de Globant. Permite realizar acciones como registrar y ver clientes, registrar Scrum Masters o programadores, y cerrar sesión, procesando la opción seleccionada por el usuario.

```
PROBLEMS 4 OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS C:\Users\usuario> & C:/Users/usuario/AppData/Local/Programs/Python/Python313/python.exe c:/Users/usuario/Music/globant.py/login.py
SISTEMA DE LOGIN
Ingresa el usuario: admin_globant
Ingresa tu contraseña: admin
Bienvenido
Bienvenido administrador Globant
1. Registrar Cliente
2. Ver Clientes
3. Registrar Scrum Master o Programador
4. Cerrar sesión
Seleccione una opción: █
```

Ejecución del terminal con ingreso de admin globant.

FUNCIONALIDAD 3 REGISTRO DE CLIENTES

```
#registro clientes
def registrar_cliente():
    usuario = input("Ingrese el usuario para el cliente: ")
    if usuario in usuarios:
        print("El usuario ya existe")
        return
    contraseña = input("Ingrese contraseña para el cliente: ")
    nombre = input("Ingrese nombre completo del cliente: ")
    cedula = input("Ingrese la cédula del cliente: ")
    telefono = input("Ingrese el teléfono del cliente: ")
    correo = input("Ingrese el correo del cliente: ")

    usuarios[usuario] = {"contraseña": contraseña, "rol": "cliente"}
    clientes.append({"usuario": usuario, "nombre": nombre, "cedula": cedula, "telefono": telefono, "correo": correo})
    print("Cliente registrado exitosamente")
```

Descripción: Se muestra la función encargada de solicitar al usuario los datos necesarios para registrar un nuevo cliente, incluyendo usuario, contraseña, nombre, cedula, teléfono y correo electrónico. Valida que el usuario no existe previamente y, si es nuevo, lo agrega a las estructuras de datos de usuarios y clientes.

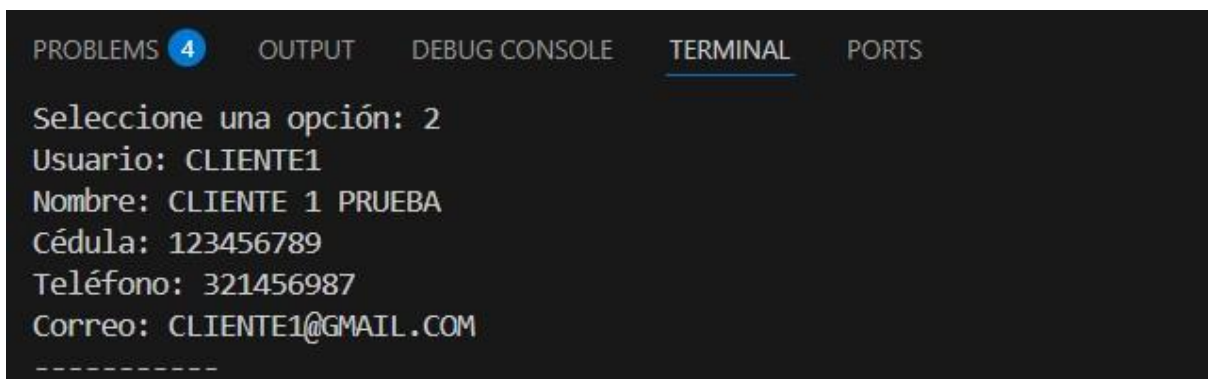
```
Seleccione una opción: 1
Ingrese el usuario para el cliente: CLIENTE1
Ingrese contraseña para el cliente: 123
Ingrese nombre completo del cliente: CLIENTE 1 PRUEBA
Ingrese la cédula del cliente: 123456789
Ingrese el teléfono del cliente: 321456987
Ingrese el correo del cliente: CLIENTE1@GMAIL.COM
Cliente registrado exitosamente
Bienvenido administrador Globant
```

Terminal en donde se evidencia el input de los datos solicitado y los datos que se ingresaron al ejecutarlo.

FUNCIONALIDAD 4 VER LOS CLIENTES

```
#ver (function) def ver_clientes() -> None
def ver_clientes():
    for cliente in clientes:
        print("Usuario:", cliente["usuario"])
        print("Nombre:", cliente["nombre"])
        print("Cédula:", cliente["cedula"])
        print("Teléfono:", cliente["telefono"])
        print("Correo:", cliente["correo"])
        print("-----")
```

Descripción: Se evidencia la función encargada o diseñada para visualizar la lista de cliente y mostrar en consola los detalles completos de cada uno, usuario, nombre, cedula, teléfono y correo electrónico.



```
PROBLEMS 4 OUTPUT DEBUG CONSOLE TERMINAL PORTS
Seleccione una opción: 2
Usuario: CLIENTE1
Nombre: CLIENTE 1 PRUEBA
Cédula: 123456789
Teléfono: 321456987
Correo: CLIENTE1@GMAIL.COM
-----
```

Consola con la visualización de los datos ingresados.

FUNCIONALIDAD 5 REGISTRAR USUARIOS (SCRUM MASTER Y PROGRAMADORES)

```
73 #registro usuarios
74 def registrar_usuario():
75     usuario = input("Ingrese el nuevo usuario: ")
76     if usuario in usuarios:
77         print("El usuario ya existe")
78         return
79     contraseña = input("Ingrese la contraseña: ")
80     print("Seleccione el rol:")
81     print("1. Programador")
82     print("2. Scrum Master")
83     opcion = input("Opción: ")
84
85     if opcion == "1":
86         rol = "desarrollador"
87     elif opcion == "2":
88         rol = "scrum_master"
89     else:
90         print("Opción inválida")
91         return
92
93     usuarios[usuario] = {"contraseña": contraseña, "rol": rol}
94     print("Usuario registrado exitosamente")
95
```

Descripción: Se evidencia la función encargada en registrar nuevos usuarios con roles específicos (Programador o Scrum Master). La función solicita el nombre de usuario, valida que no exista, pide una contraseña y permite seleccionar el rol antes de registrar el usuario.

```
Seleccione una opción: 3
Ingrese el nuevo usuario: SCRUM8
Ingrese la contraseña: 123
Seleccione el rol:
1. Programador
2. Scrum Master
Opción: 2
Usuario registrado exitosamente
```

Ejecutando la consola registrando la opción 2 de Scrum Master.

```

Seleccione una opción: 3
Ingrese el nuevo usuario: PROGRAMADOR8
Ingrese la contraseña: 123
Seleccione el rol:
1. Programador
2. Scrum Master
Opción: 1
Usuario registrado exitosamente
Bienvenido administrador Globant

```

Ejecutando la consola registrando la opción 1 que corresponde a Programador.

FUNCIONALIDAD 6 MENU DEL CLIENTE

```

#Menú Cliente
def menu_cliente(usuario):
    while True:
        print("Bienvenido", usuario)
        print("1. Escriba lo que desea que nuestra empresa Globant haga para usted")
        print("2. Cerrar sesión")

        opcion = input("Opción: ")

        if opcion == "1":
            comentario = input("Escriba su comentario: ")
            comentarios[usuario] = comentario
            print("Comentario guardado")
        elif opcion == "2":
            break
        else:
            print("Opción no válida")

```

Descripción: Se evidencia la función de un menú interactivo al cliente, permitiéndole enviar comentarios sobre los servicios de la empresa Globant o cerrar su sesión. Los comentarios se guardan asociados al usuario.

```

Ingresa el usuario: cliente1
Ingresa tu contraseña: 123
Bienvenido
Bienvenido cliente1
1. Escriba lo que desea que nuestra empresa Globant haga para usted
2. Cerrar sesión
Opción: 

```

Ejecución de consola como cliente abriendo el menú para dejar comentarios hacia la empresa.

FUNCIONALIDAD 7 MENU SCRUM MASTER

```
#Menú Scrum Master
def menu_scrum(usuario):
    while True:
        print("Bienvenido Scrum Master")
        print("1. Ver comentarios de clientes")
        print("2. Asignar proyecto a desarrollador")
        print("3. Ver novedades de los desarrolladores")
        print("4. Cerrar sesión")

        opcion = input("Opción: ")

        if opcion == "1":
            for cliente, texto in comentarios.items():
                print("Cliente:", cliente)
                print("Comentario:", texto)
                print("-----")
        elif opcion == "2":
            cliente = input("Ingrese usuario del cliente: ")
            desarrollador = input("Ingrese usuario del programador: ")
            descripcion = input("Describa el trabajo a asignar: ")
            proyectos[desarrollador] = {"cliente": cliente, "trabajo": descripcion}
            print("Trabajo asignado")
        elif opcion == "3":
            for dev, texto in novedades.items():
                print("Desarrollador:", dev)
                print("Novedad:", texto)
        elif opcion == "4":
            break
        else:
            print("Opción no válida")
```

Descripción: Se evidencia la función que ofrece un menú interactivo al Scrum Master, que permite ver comentarios de clientes, asignar proyectos a desarrolladores y ver novedades de los desarrolladores, gestionado cada acción según la opción seleccionada.

```
Bienvenido Scrum Master
1. Ver comentarios de clientes
2. Asignar proyecto a desarrollador
3. Ver novedades de los desarrolladores
4. Cerrar sesión
Opción: █
```

Ejecución de consola del Scrum Master, con su respectivo menú.

FUNCIONALIDAD 8 MENU PROGRAMADOR

```
#Menú Programador
def menu_programador(usuario):
    while True:
        print("Bienvenido Programador")
        print("1. Ver trabajo asignado")
        print("2. Enviar novedad al Scrum Master")
        print("3. Cerrar sesión")

        opcion = input("Opción: ")

        if opcion == "1":
            if usuario in proyectos:
                trabajo = proyectos[usuario]
                print("Cliente:", trabajo["cliente"])
                print("Trabajo:", trabajo["trabajo"])
            else:
                print("No tiene trabajo asignado")
        elif opcion == "2":
            novedad = input("Escriba su novedad: ")
            novedades[usuario] = novedad
            print("Novedad enviada")
        elif opcion == "3":
            break
        else:
            print("Opción no válida")
```

Descripción: Se evidencia la función que ejecuta el menú interactivo al programador, el cual permite ver el trabajo asignado, enviar novedades al Scrum Master o cerrar sesión, gestionando cada acción según la opción seleccionada.

```
Bienvenido Programador
1. Ver trabajo asignado
2. Enviar novedad al Scrum Master
3. Cerrar sesión
Opción: █
```

Consola del menú del programador.

SISTEMA PRINCIPAL Y CAMBIO DE USUARIO

```
#Sistema Principal
while True:
    print("SISTEMA DE LOGIN")
    rol, usuario = login()

    if rol == "admin":
        menu_administrador()
    elif rol == "cliente":
        menu_cliente(usuario)
    elif rol == "scrum_master":
        menu_scrum(usuario)
    elif rol == "desarrollador":
        menu_programador(usuario)

    # Pregunta si desea salir o cambiar de usuario
    while True:
        print("¿Desea salir del sistema completamente?")
        print("1. Sí, salir")
        print("2. No, cambiar de usuario")
        opcion = input("Opción: ")
        if opcion != "1":
            print("Hasta luego")
        else:
            break
```

Descripción: Se evidencia la función principal del sistema, que representa el punto de entrada y control de la aplicación. El bucle asegura que el sistema permanezca activo. Redirige a un menú específico según el rol (admin, cliente, scrum master y desarrollador), y al finalizar la sesión, se encuentra la opción de salir completamente del sistema o cambiar de usuario.

3.4 Desarrollo de la propuesta en Java

PATRON BUILDER

El Builder aparece en el código a través de clases como:

- ConstructorTareaGlobant
- El proceso de construcción paso a paso:
 - nombreTarea()
 - detalle()
 - prioridad()

```
public class ConstructorTareaGlobant {  
    private TareaGlobant tarea = new TareaGlobant();  
  
    public ConstructorTareaGlobant conNombre(String nombre) {  
        tarea.nombreTarea = nombre;  
        return this;  
    }  
  
    public ConstructorTareaGlobant conDetalle(String detalle) {  
        tarea.detalle = detalle;  
        return this;  
    }  
  
    public ConstructorTareaGlobant conPrioridad(int prioridad) {  
        tarea.nivelPrioridad = prioridad;  
        return this;  
    }  
  
    public TareaGlobant construir() {  
        return tarea;  
    }  
}
```

Justificación

La clase ConstructorTareaGlobant sirve para crear objetos de TareaGlobant paso a paso. Permite definir el nombre, el detalle y la prioridad usando métodos simples, y cada uno devuelve el mismo constructor para poder encadenarlos de forma cómoda. Esto evita errores y también la necesidad de tener muchos constructores diferentes. La idea es separar el proceso de creación del objeto de

su resultado final. Este patrón es muy útil cuando una tarea necesita varios datos que se van definiendo poco a poco y puede armarse de diferentes maneras según lo que se necesite.

PATRON FACTORY

```
public class FabricaUsuarios {  
    public static PersonaGlobant crearUsuario(String nombre, String clave, RolGlobant rol) {  
        switch (rol) {  
            case CLIENTE: return new ClienteGlobant(nombre, clave);  
            case LIDER: return new LiderGlobant(nombre, clave);  
            case PROGRAMADOR: return new ProgramadorGlobant(nombre, clave);  
            case ADMINISTRADOR:  
            default:  
                return new PersonaGlobant(nombre, clave, RolGlobant.ADMINISTRADOR);  
        }  
    }  
}
```

Explicación

La clase FabricaUsuarios usa el patrón Factory Method para hacer más fácil y ordenada la creación de usuarios. Básicamente, esta fábrica se encarga de decidir qué tipo de usuario crear según el rol: cliente, líder, programador o administrador. Todo esto se hace desde un solo lugar y sin que otras partes del sistema tengan que preocuparse por cómo se crean esos objetos. Todos los usuarios que genera vienen de la clase base PersonaGlobant, así que comparten la misma estructura principal.

PATRON SIGLETON

```

package GlobantProject;

import javax.swing.JOptionPane;

// Singleton: solo existe una instancia de ServicioAuthGlobant
public class ServicioAuthGlobant {
    private static ServicioAuthGlobant instancia; // instancia única

    // Constructor privado: evita crear nuevas instancias
    private ServicioAuthGlobant() {}

    // Método para obtener la instancia única
    public static ServicioAuthGlobant obtenerInstancia() {
        if (instancia == null) instancia = new ServicioAuthGlobant();
        return instancia;
    }

    // Método para iniciar sesión (dependencia: usa FabricaUsuarios)
    public PersonaGlobant iniciarSesion() {
        String usuario = JOptionPane.showInputDialog(message:"Ingresa tu usuario:");
        String clave = JOptionPane.showInputDialog(message:"Ingresa tu clave:");

        if(usuario.equals(anObject:"admin") && clave.equals(anObject:"admin")) {
            return FabricaUsuarios.crearUsuario(nombre: usuario, clave, rol:RolGlobant.ADMINISTRADOR);
        }
        if(usuario.equals(anObject:"lider") && clave.equals(anObject:"123")) {
            return FabricaUsuarios.crearUsuario(nombre: usuario, clave, rol:RolGlobant.LIDER);
        }
        if(usuario.equals(anObject:"dev") && clave.equals(anObject:"123")) {
            return FabricaUsuarios.crearUsuario(nombre: usuario, clave, rol:RolGlobant.PROGRAMADOR);
        }
        if(usuario.equals(anObject:"cliente") && clave.equals(anObject:"123")) {
            return FabricaUsuarios.crearUsuario(nombre: usuario, clave, rol:RolGlobant.CLIENTE);
        }

        JOptionPane.showMessageDialog(parentComponent:null, message:"Credenciales incorrectas");
        return null;
    }
}

```

Explicación

La clase ServicioAuthGlobant usa el patrón Singleton, que sirve para asegurar que solo exista una única instancia durante toda la ejecución del programa. Para conseguirlo, tiene un atributo estático llamado instancia y un constructor privado, lo que impide que se creen objetos desde fuera.

El método obtenerInstancia() se encarga de crear el servicio solo cuando hace falta y luego reutilizarlo. Esta clase concentra todo el proceso de autenticación: pide las credenciales y verifica al usuario apoyándose en la clase FabricaUsuarios, que se encarga de construir el objeto correcto según el rol.

El patrón Singleton es ideal en este caso porque el manejo de acceso debe controlarse desde un solo punto, evitando problemas como sesiones duplicadas o información inconsistente.

Al devolver distintos tipos de usuarios dependiendo de los datos ingresados, esta clase trabaja en conjunto con el polimorfismo y mantiene organizada la lógica general del sistema.

CLASE PRINCIPAL

```
public class InicioGlobant {  
    public static void main(String[] args) {  
        new MenuGlobant().iniciar();  
    }  
}
```

Explicación

Esta clase funciona como el punto de inicio de la aplicación, usando el método `main(String[] args)` que Java requiere. Su trabajo principal es arrancar el programa y poner en marcha todo el flujo del sistema de forma clara y ordenada, sin mezclarse con otras responsabilidades. Al actuar solo como el “lanzador” de la aplicación, InicioGlobant garantiza que todo empiece de manera limpia y organizada. Gracias a esto, el proyecto mantiene una estructura modular donde cada clase sabe exactamente qué debe hacer.

HERENCIA, POLIMORFISMO Y ENCAPSULAMIENTO

Clase ClienteGlobant (Herencia y Polimorfismo)

```

public class ClienteGlobant extends PersonaGlobant {
    public ClienteGlobant(String nombre, String clave) {
        super(nombre, clave, RolGlobant.CLIENTE);
    }

    @Override
    public void mostrarAcceso() {
        System.out.println("Acceso como Cliente.");
    }
}

```

Explicación

La clase ClienteGlobant usa herencia y polimorfismo para representar a los usuarios tipo cliente dentro del sistema. Como extiende de PersonaGlobant, aprovecha sus atributos y métodos, evitando repetir código y manteniendo todo consistente.

En su constructor, esta clase asigna automáticamente el rol de CLIENTE, así cada objeto creado queda identificado correctamente.

Además, sobrescribe el método mostrarAcceso() para mostrar un mensaje propio del cliente. Gracias al polimorfismo, aunque el objeto se trate como una PersonaGlobant, el sistema ejecuta la versión correcta del método según el tipo real del usuario. Esto permite que el comportamiento sea el adecuado para cada rol.

Clase LiderGlobant (Polimorfismo)

```

public class LiderGlobant extends PersonaGlobant {
    public LiderGlobant(String nombre, String clave) {
        super(nombre, clave, RolGlobant.LIDER);
    }

    @Override
    public void mostrarAcceso() {
        System.out.println("Acceso como Líder de Proyecto.");
    }
}

```

Explicación

La clase LiderGlobant es parte de la jerarquía de usuarios del sistema y usa herencia y polimorfismo como base. Al extender de PersonaGlobant, hereda datos y funciones como el nombre, la clave y el rol, lo que evita repetir código y hace el diseño más ordenado.

En su constructor, esta clase asigna automáticamente el rol LIDER, asegurando que cada objeto creado quede bien identificado dentro del sistema. Esto ayuda a mantener reglas claras para cada tipo de usuario.

En general, LiderGlobant aporta a un diseño modular y fácil de mantener, permitiendo añadir nuevos roles más adelante sin tener que modificar lo que ya existe.

Clase MenuGlobant (Polimorfismo Dinámico)

```

public class MenuGlobant {
    public void iniciar() {
        PersonaGlobant persona;

        // Loop hasta que se loguee correctamente
        do {
            persona = ServicioAuthGlobant.obtenerInstancia().iniciarSesion(); // Singleton + Factory
        } while (persona == null);

        // Polimorfismo: se ejecuta el mostrarAcceso() correspondiente al tipo real de usuario
        persona.mostrarAcceso();

        mostrarMenu(persona);
    }

    private void mostrarMenu(PersonaGlobant persona) {
        int opcion = -1;

        do {
            try {
                opcion = Integer.parseInt(JOptionPane.showInputDialog(
                    "=== MENÚ GLOBANT ===\n" +
                    "1. Crear tarea\n" +
                    "2. Ver tarea\n" +
                    "0. Salir"
                ));

                switch (opcion) {
                    case 1 -> crearTarea();
                    case 2 -> JOptionPane.showMessageDialog(null, "Aún no hay tareas guardadas.");
                }
            } catch (NumberFormatException e) {
                JOptionPane.showMessageDialog(null, "Ingresa un número válido");
            }
        } while (opcion != 0);
    }
}

```

```

private void crearTarea() {
    String nombre = JOptionPane.showInputDialog("Nombre de la tarea:");
    String detalle = JOptionPane.showInputDialog("Detalle de la tarea:");
    int prioridad = Integer.parseInt(JOptionPane.showInputDialog("Prioridad (1-5):"));

    // Builder: construye la tarea paso a paso
    TareaGlobant tarea = new ConstructorTareaGlobant()
        .conNombre(nombre)
        .conDetalle(detalle)
        .conPrioridad(prioridad)
        .construir();

    JOptionPane.showMessageDialog(null, "Tarea creada:\n" + nombre);
    tarea.mostrar();
}

```

Explicación

La clase MenuGlobant actúa como el controlador principal que se encarga de la interacción con el usuario. Su trabajo es manejar el flujo del programa después de que alguien inicia sesión, mostrar las opciones disponibles y coordinar la creación o visualización de tareas.

Para funcionar, la clase depende del servicio de autenticación (ServicioAuthGlobant), que usa el patrón Singleton para asegurarse de que solo exista una única instancia encargada de validar los datos del usuario. El método iniciar() utiliza este servicio para realizar el inicio de sesión y también aplica el patrón Factory, ya que el servicio devuelve distintos tipos de usuarios según el rol ingresado (cliente, líder, programador, etc.).

En general, MenuGlobant es una pieza clave del sistema, porque une varios conceptos y patrones de diseño para mantener un flujo de trabajo claro, organizado y fácil de ampliar.

Clase PersonaGlobant (Herencia y Encapsulamiento)


```

package GlobantProject;

// Clase base de todos los usuarios
// Conceptos: Herencia, Encapsulamiento
public class PersonaGlobant {
    protected String nombreUsuario;    // Encapsulamiento parcial (protected)
    protected String claveSecreta;
    protected RolGlobant tipoRol;

    // Constructor
    public PersonaGlobant(String nombreUsuario, String claveSecreta, RolGlobant tipoRol) {
        this.nombreUsuario = nombreUsuario;
        this.claveSecreta = claveSecreta;
        this.tipoRol = tipoRol;
    }

    // Getter del rol
    public RolGlobant obtenerRol() { return tipoRol; }

    // Getter del nombre
    public String obtenerNombre() { return nombreUsuario; }

    // Método que será sobrescrito por las subclases (Polimorfismo)
    public void mostrarAcceso() {
        System.out.println(x: "Acceso básico de usuario.");
    }

    // Ejemplo de sobrecarga de método (no obligatorio, pero útil para POO)
    public void mostrarAcceso(String mensajeExtra) {
        System.out.println(mensajeExtra + " - Usuario: " + nombreUsuario);
    }
}

```

Explicación

La clase `PersonaGlobant` funciona como la base de todos los tipos de usuarios del sistema. Es una superclase, lo que significa que otras clases como `ClienteGlobant`, `LiderGlobant` y `ProgramadorGlobant` toman sus características y las amplían. Aquí se define información esencial: el nombre, la contraseña y el rol. Estos datos están protegidos con el modificador `protected`, lo que permite que solo las clases hijas o las que estén en el mismo paquete puedan acceder a ellos directamente. Esto ayuda a mantener un buen control sobre la información.

Además, la clase ofrece métodos públicos para consultar datos importantes, como el nombre o el rol. También incluye el método `mostrarAcceso()`, pensado para que las clases hijas lo sobrescriban y muestren su propio comportamiento según el tipo de usuario.

En conjunto, `PersonaGlobant` establece una estructura clara, flexible y lista para crecer, permitiendo organizar y manejar distintos perfiles dentro del sistema sin duplicar código.

Clase ProgramadorGlobant (Polimorfismo)

```
package GlobantProject;

// ProgramadorGlobant hereda de PersonaGlobant
// Polimorfismo
public class ProgramadorGlobant extends PersonaGlobant {
    public ProgramadorGlobant(String nombre, String clave) {
        super(nombreUsuario: nombre, claveSecreta: clave, tipoRol: RolGlobant.PROGRAMADOR);
    }

    @Override
    public void mostrarAcceso() {
        System.out.println(x: "Acceso como Programador.");
    }
}
```

Explicación

La clase ProgramadorGlobant representa a un usuario especializado dentro del sistema y muestra muy bien cómo funciona el polimorfismo. Al extender la clase PersonaGlobant, hereda toda la información y las acciones básicas, pero modifica el método mostrarAcceso() para mostrar un mensaje propio, pensado únicamente para los programadores.

Este comportamiento es un ejemplo directo de polimorfismo dinámico: aunque el objeto pueda tratarse como una PersonaGlobant, en tiempo de ejecución se ejecuta la versión del método que corresponde realmente al tipo ProgramadorGlobant.

Además, al crear una instancia de esta clase, se asigna automáticamente el rol PROGRAMADOR usando la enumeración RolGlobant. Aunque su estructura es simple, cumple una función importante: permite que el sistema distinga a un programador del resto de usuarios y muestre el acceso adecuado cuando inicia sesión.

Clase RolGloant (Enumeración)

```
package GlobantProject;

// Enum que define los tipos de rol de usuario
// Concepto POO: Enumeración
public enum RolGlobant {
    ADMINISTRADOR,
    LIDER,
    PROGRAMADOR,
    CLIENTE
}
```

Explicación

La clase RolGlobant es una enumeración (enum), una herramienta muy útil en programación orientada a objetos que permite definir un conjunto de valores fijos. En este sistema, los roles posibles son: ADMINISTRADOR, LIDER, PROGRAMADOR y CLIENTE.

Usar un enum ayuda a evitar errores comunes, como escribir mal un rol o usar valores que no existen. Además, garantiza que solo se puedan seleccionar opciones válidas, lo que mantiene el programa ordenado y consistente.

Esta enumeración es esencial porque el sistema necesita identificar correctamente el rol de cada usuario para darle el comportamiento adecuado en las clases que heredan de PersonaGlobant.

También es utilizada por FabricaUsuarios cuando crea nuevos objetos según el rol asignado.

Al colocar todos los roles en un mismo lugar, el código se vuelve más fácil de entender, modificar y mantener, evitando tener constantes repetidas en diferentes partes del proyecto.

Clase TareaGlobant (Encapsulamiento)

```

package GlobantProject;

// Clase que representa una tarea
// Concepto POO: Encapsulamiento
public class TareaGlobant {
    protected String nombreTarea;
    protected String detalle;
    protected int nivelPrioridad;

    // Método para mostrar información de la tarea
    public void mostrar() {
        System.out.println("Tarea: " + nombreTarea);
        System.out.println("Detalle: " + detalle);
        System.out.println("Prioridad: " + nivelPrioridad);
    }
}

```

Explicación

La clase TareaGlobant define la forma básica que tendrá una tarea dentro del sistema y aplica el principio de encapsulamiento. Sus atributos nombreTarea, detalle y nivelPrioridad están marcados como protected, lo que evita que cualquier clase externa pueda modificarlos directamente.

Esto permite un mejor control sobre la información y previene cambios no deseados. Aunque no son privados, este nivel de protección facilita que ConstructorTareaGlobant pueda crear tareas paso a paso usando el patrón Builder, accediendo a los campos sin romper la organización interna del paquete.

Además, la clase incluye el método mostrar(), que presenta los datos de manera clara y segura. En conjunto, TareaGlobant funciona como un contenedor bien estructurado que separa la información de la tarea de su proceso de construcción y de la interfaz, ayudando a mantener un sistema modular y fácil de mantener.

DIAGRAMAS

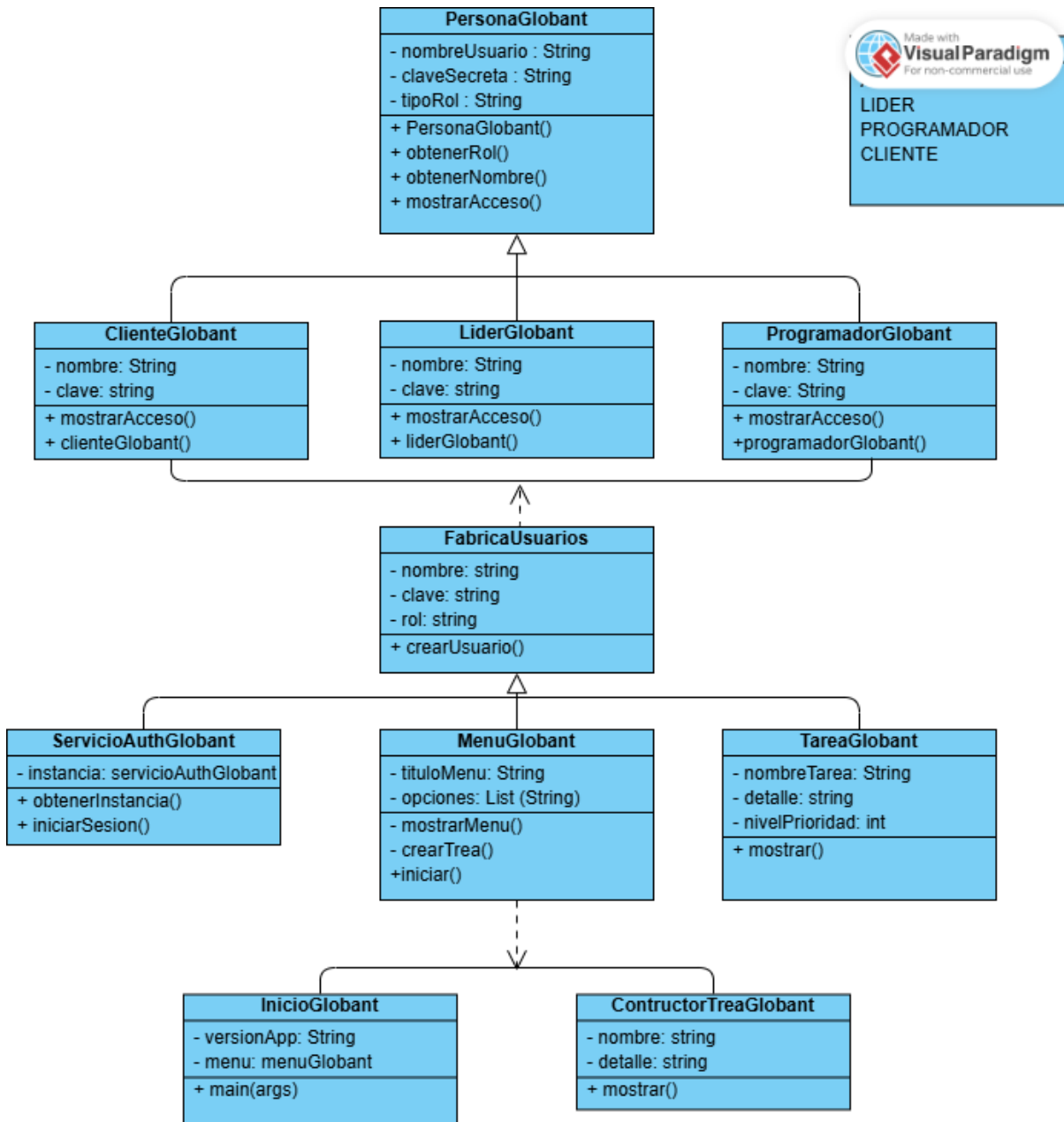


Ilustración 1 Diagrama de Clases

Nota: Diagrama propio elaborado en Visual Paradigm.

Diagrama de estado TareaGlobant

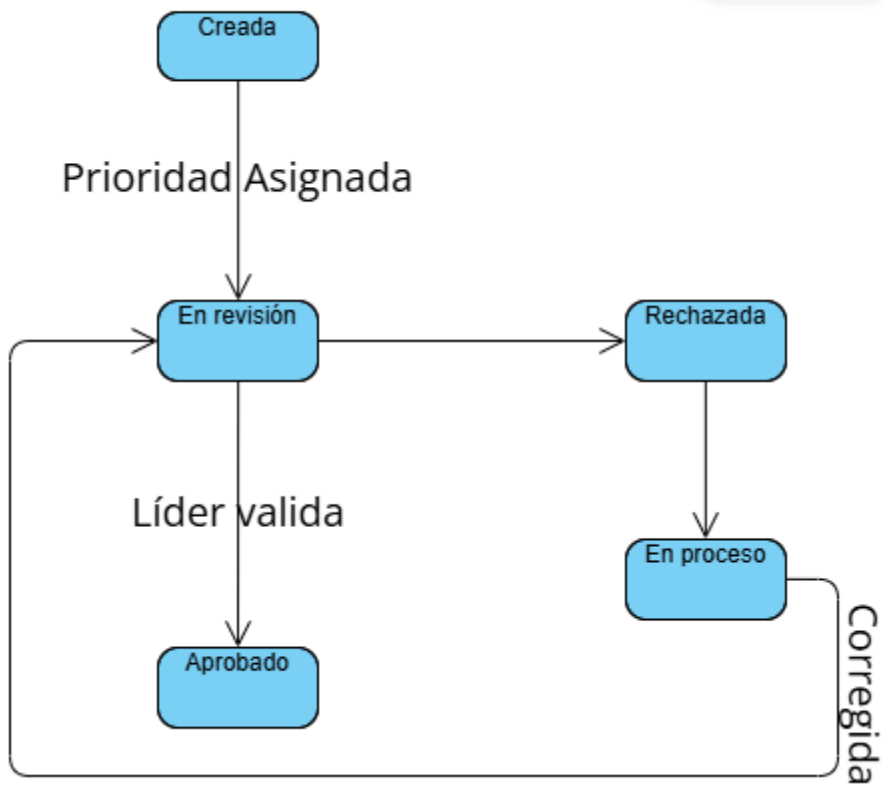


Ilustración 2 Diagrama de Estados - TareaGlobant

Nota: Diagrama propio elaborado en Visual Paradigm.

Diagrama de estado ServicioAuthGlobant

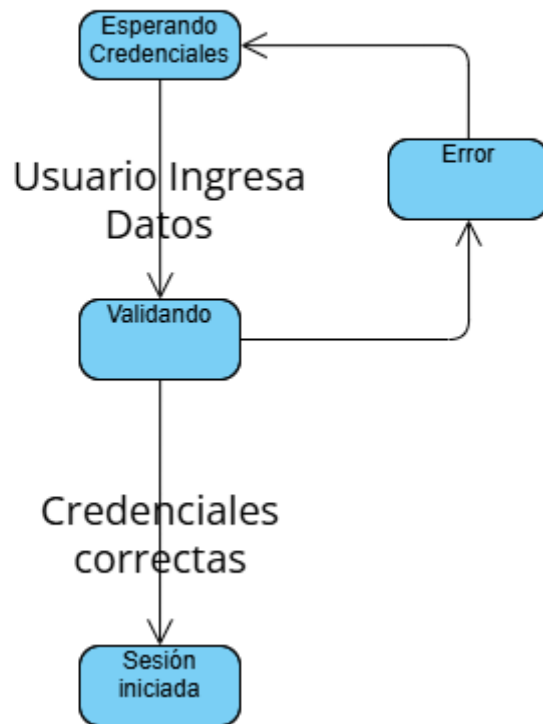


Ilustración 3 Diagrama de Estados - ServicioAuthGlobant

Nota: Diagrama propio elaborado en Visual Paradigm.

Diagrama de uso del Sistema de Globant

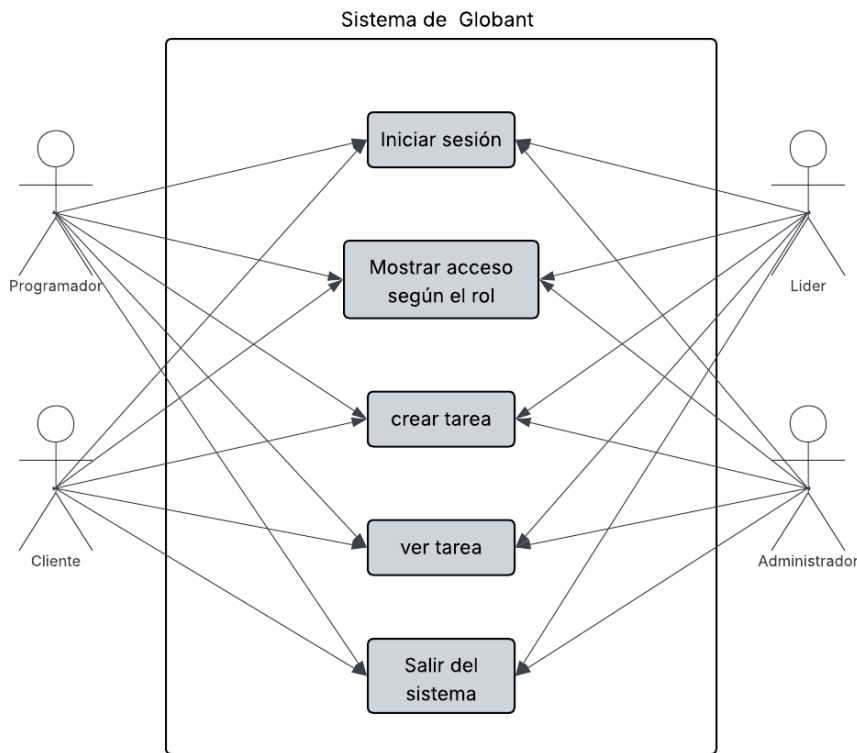


Ilustración 4 Diagrama de Uso – Sistema de Globant

Nota: Diagrama propio elaborado en Lucidapp

Diagrama de Secuencia

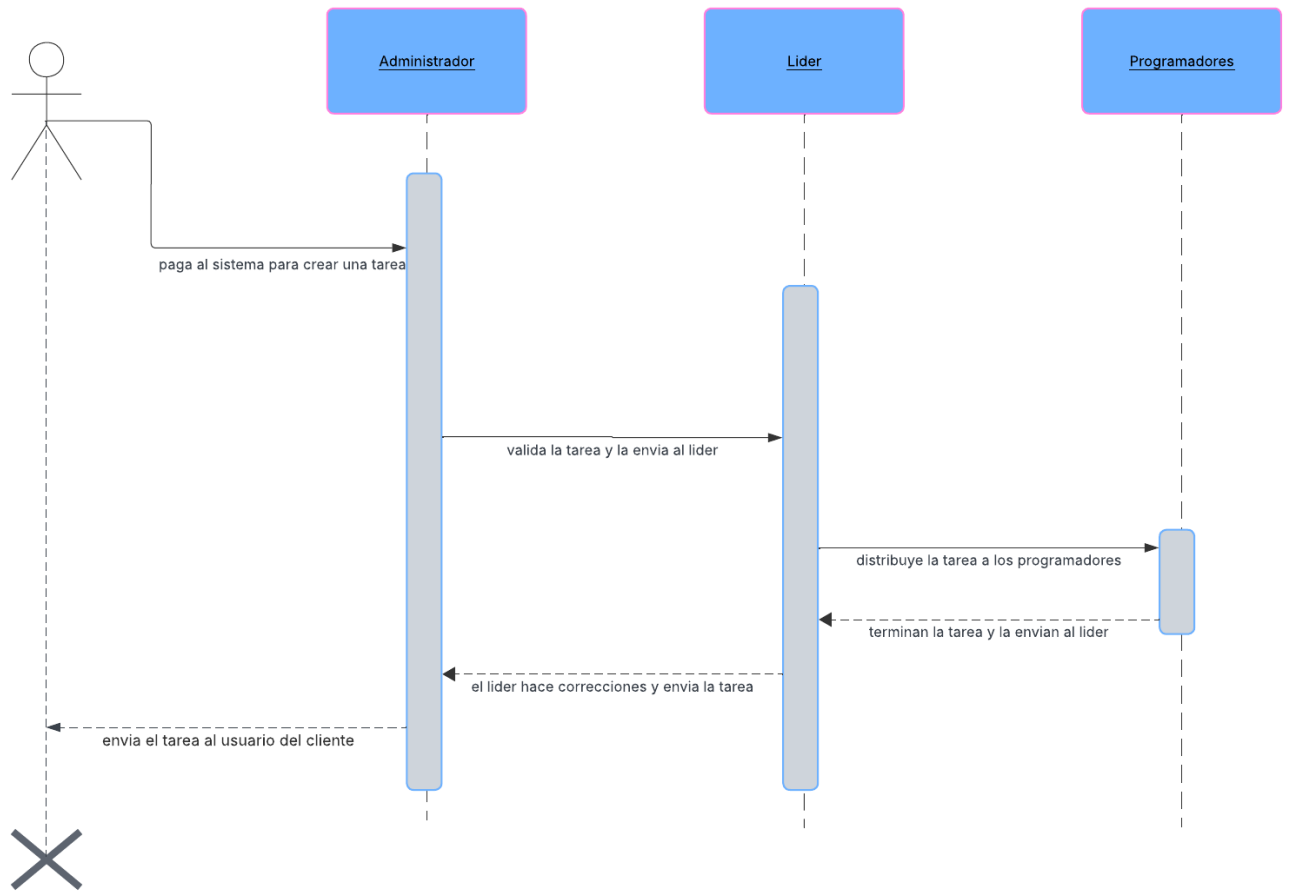


Ilustración 5 Diagrama de secuencia – Proceso imaginario de tareas

Nota: Diagrama propio elaborado en Lucidapp

1.1. Conclusiones

- El código presenta un sistema de gestión de usuarios y clientes con una estructura clara y funcionalidades diferenciadas por rol, centrado en la autenticación, asignación de tareas y comunicación interna.
- El sistema desarrollado integra de manera efectiva principios fundamentales de la programación orientada a objetos y patrones de diseño que fortalecen su estructura y funcionamiento. La aplicación de patrones como Builder y Factory facilita la creación organizada y flexible de objetos, mientras que la herencia y el polimorfismo permiten un comportamiento más dinámico y una arquitectura fácilmente extensible. Además, la organización clara del flujo de la aplicación y la separación adecuada de responsabilidades contribuyen a un diseño modular, mantenible y escalable. En conjunto, estos elementos permiten construir un sistema coherente, robusto y preparado para futuras ampliaciones o mejoras.

2. Referencias

Angelini, A. (2021). *Cómo usar las metodologías ágiles en tu equipo según la líder de la práctica de agilidad en Globant*. Obtenido de Endeavor Hub: <https://endeavor-hub.com/hub-article-como-usar-las-metodologias-agiles-en-tu-equipo-segun-la-lider-de-la-practica-de-agilidad-en-globant-2/>

Atlassian. (s.f.). *What is Trello?* Obtenido de Trello: <https://trello.com/platforms>

Atlassian. (s.f.). *Jira Software Features*. Obtenido de Atlassian: <https://www.atlassian.com/software/jira/features>

ClickUp. (s.f.). *Features | ClickUp™*. Obtenido de ClickUp: <https://clickup.com/features>

Evans, Z. &. (2017). *ClickUp: Productivity Platform*. Obtenido de ClickUp: <https://clickup.com/about>

Globant. (26 de 11 de 2021). *El arte de facilitar el cambio*. Obtenido de Blog Globant: <https://stayrelevant.globant.com/es/technology/agile-organizations/arte-facilitar-cambio/>

Globant. (2021). *Experiencia convergente y aceleradores digitales, los motores de la transformación digital*. Obtenido de Blog Globant: https://statics.globant.com/production/public/2021-05/Globant_Case_Study_Telecom_Es.pdf

Globant. (30 de 08 de 2021). *Nuestra capacidad de cumplir*. Obtenido de Blog Globant: <https://stayrelevant.globant.com/es/technology/agile-organizations/nuestra-capacidad-de-cumplir/>

Globant. (2022). *Agile Days 2022*. Obtenido de Blog Globant:

<https://more.globant.com/es/agile-days-2022>

Globant. (2023). *Cultura organizacional y consultoría ágil*. Obtenido de Blog Globant:

<https://www.globant.com/es/studio/cultural-hacking-and-agility>

Globant. (06 de 09 de 2024). *Metodologías ágiles de desarrollo de software*. Obtenido de

Blog Globant: <https://stayrelevant.globant.com/es/technology/agile-organizations/>

Globant. (2025). *Carreras*. Obtenido de Globant: <https://www.globant.com/es/careers>

Globant. (2025). *Programa Code Your Future construye paz*. Obtenido de Globant:

<https://www.globant.com/es/programa-code-your-future-construye-paz>

Globant. (2025). *Sobre Nosotros*. Obtenido de Globant:

<https://www.globant.com/es/aboutGlobant>